Last updated on Nov. 9, 2026

Sangryul Kim

Seoul, Republic of Korea ksr970330@gmail.com sangryul.github.io linkedin.com/in/sangryul-kim

Summary

I design and build agentic AI systems. Given the result someone wants, I can quickly work out an architecture that delivers it, build a version they can try, and keep reshaping it from their feedback. My path has run through most layers of these systems: retrieval and retrieval-augmented generation (RAG) research at KAIST, an agentic search pipeline at LINER, a Korean embedding model at NAVER Search, and now, as a Machine Learning Engineer at NAVER, LLM evaluation systems and agent prototypes for the content feeds in the NAVER app. I pick the architecture to fit the problem: one judge with a strict rubric where the rules are clear, a planner held in check by code and scored by a blind judge where the model has room to choose, and several cooperating agents where a single viewpoint is not enough. Many of the people I build for do not write code, so I start from the outcome they care about and measure whether the system reaches it. M.S. in AI from KAIST, with first-author papers at COLING 2025 and Findings of ACL 2025. Native Korean speaker, fluent in English.

GenAI Engineering Practice

I choose an agent architecture for the task instead of starting from a framework, and I write the orchestration in plain Python. In most of my designs the LLM reads and plans while code enforces: plans come back in a fixed JSON schema with hard limits, code rejects anything outside the allowed set, and a separate judge scores the outcome without knowing which strategy produced it. When one perspective is not enough, I split the work across agents, for example persona agents that each read an A/B test for one user segment and a synthesizer that combines them.

Retrieval is where I started. I researched RAG at KAIST, designed a pipeline at LINER in which an agent decides what to search for and gathers the results, and ran reranker experiments there. At NAVER Search I trained a Korean embedding model on ModernBERT with contrastive learning followed by fine-tuning, and in the NAVER app I build retrievers for the content feeds.

I treat evaluation as part of the design. Each surface gets its own rubric, the judge's verdicts drive removal or demotion, and the policy team's labels and reviews flow back into versioned rubrics. I compare judge models against labeled sets with Cohen's kappa and bootstrap confidence intervals, fixing the metric and threshold before the run. What the system catches automatically, by surface:

- Topic feeds: exposure of articles marked for removal fell by 93.7 to 99.9% over a 159-hour A/B test.
- Home feed: held-out recall rose from 38% to 90% for low-quality entertainment content and from 2% to 68% for sensational content; a later version traded some of that recall for almost no false alarms.
- Clickbait: recall rose from 3% to 69% after benchmarking four models on one labeled set.

For serving, I moved judging from a commercial LLM API to a self-hosted vision-language model, and I plan GPU capacity from measured throughput. Outputs follow a JSON schema, and the pipeline fails open, so an article without a verdict passes through and a stalled judge cannot empty the feed. Feeding images along with the text removed most of a bias against video articles.

Claude Code is my main development environment, with Codex as a second reviewer. I wrote my own skills for it, including a review in which a reviewer subagent and another vendor's model both check a change. Agents that run unattended get an allowlist of tools and download domains. I also started an internal thread on AI coding tools and wrote an onboarding guide that a teammate's coding agent can read to set up the same environment.

Experience

NAVER, Machine Learning Engineer, Agentic App AI team — Seongnam-si, Republic of Korea — Jul. 2025 to Present

- Designed and launched an hourly pipeline in which a vision-language model reviews articles for the NAVER app's topic feeds, reading the text together with its images, and the feed server uses the scores to remove or demote low-quality articles. I rolled it out to all traffic after an A/B test, then moved judging off a commercial LLM API and onto a self-hosted open-weight model.

- Rebuilt the always-on service that watches the home feed for harmful and low-quality content and emails alerts to the content-policy and quality teams. I tune its rubric against their labeled sets and manual reviews, and tell them plainly when a change cuts false alarms at the cost of recall.
- Launched the 2026 FIFA World Cup topic feed on schedule across four systems, coordinating with several partner teams; it included a retriever built on NAVER's live-streaming service. We reused the same setup for the Asian Games.
- Ran ranking experiments on the topic feeds. A signal that demotes articles whose click-through rate suddenly drops lost its first A/B test, won the next two and went to all users.
- Mentored three interns on an LLM evaluation project and have reviewed more than 130 pull requests for the team.

LINER, AI Engineer — Seoul, Republic of Korea — Mar. 2025 to Jun. 2025

- Designed a pipeline for LINER's agent search service in which an LLM agent decides what to search for and gathers the results it needs to answer a question.
- Ran reranker experiments on the retrieved results.
- Added an automated pipeline that evaluates the quality of search results, so that we could check changes to retrieval and ranking without manual review.

NAVER Search, Machine Learning Engineer, Intern — Seongnam-si, Republic of Korea — Jan. 2025 to Feb. 2025

- Trained a Korean text embedding model on ModernBERT in two stages, contrastive learning first and fine-tuning after, and it clearly outperformed the existing baselines.

NAVER WEBTOON, Machine Learning Engineer, Intern — Seongnam-si, Republic of Korea — Jul. 2022 to Oct. 2022

- Developed an end-to-end program that blocks inappropriate ads by analyzing and visualizing data from ad images.
- Worked on data preprocessing and statistical machine learning, and deployed and maintained the data and model.

Cyber Operations Command, Republic of Korea, Software Developer (mandatory military service, discharged as sergeant) — Seoul, Republic of Korea — Feb. 2019 to Sep. 2020

- Developed and operated a cyber-attack simulation program for military training, written in C++ with the Windows API.
- Built simulated malware and a command-and-control server for defensive training exercises, using C++ and Python.
- Developed a mobile security diagnosis system with MetaDefender, ADB, Vue.js and Node.js.

ONYCOM Inc, Software Engineer — Seoul, Republic of Korea — Jan. 2018 to Jan. 2019

- Developed the web service for the pre-release version of IMQA, a mobile performance monitoring and crash reporting product.

Projects

Agentic feed prototypes, NAVER — 2026

- A session strategist. An LLM agent reads a user's replayed home-feed session and proposes recipes for recomposing it, and a blind LLM judge scores each recipe on seven axes. Underneath is a diagnostic tool I designed to explain a session rather than predict clicks. An autonomous test run caught the agent overusing some levers, and I fixed that with limits in the output schema.
- A retriever selector running on a copy of the feed server. The LLM strategist writes a plan stating the user's interests, what their feed is missing, which retrievers to

boost or switch off, what to watch and when to roll back. Code guards check the plan against the candidate list and per-action limits, and a two-stage reasoning budget reached the same decisions as an unlimited one in a fraction of the time.

- Research on reading A/B tests with multiple agents. Persona agents, each built from a cluster of real user behavior, read the results from their segment's point of view, and a synthesizer agent combines them. In our own experiments the framework caught cases of Simpson's paradox that a single overall readout would have missed.
- A natural-language preference demo. Users describe the feed they want, the LLM turns the request into structured interests, and code reranks the session; a one-tap mode skips the LLM call entirely.

Gompada / github 2022

- Developed Gompada, an AI tool that grades descriptive answers using SBERT and FastText, based on keyword matching and contextual similarity.
- Designed and built the front end (Vue.js) and back end (FastAPI), and cut response time by changing how the ML models load.

Help Diana / github / demo 2021

- Developed a web service that uses AI to help patients understand what their doctors tell them, built on a Vue.js PWA, Spring, MySQL, AWS and Docker.
- Led planning and design, and handled front-end development, AWS setup, data crawling and API integration, including OCR and tokenization of medical terms.

AUTO TA / github 2021

- Developed a stock trading prediction project using FastAI, TensorFlow, PyTorch, FastAPI and Docker.
- Led planning, model design, server development and data collection, and ran experiments with CNN and LSTM models for real-time trading strategies.

Skills

Languages: Python, SQL, TypeScript, JavaScript, C++, Java
Machine learning and LLMs: PyTorch, Hugging Face, vLLM, LLM-as-a-judge, vision-language models, prompt optimization, distillation, structured outputs, retrieval, text embeddings, reranking, RAG
Models: self-hosted open-weight LLMs and vision-language models (Qwen family, Gemma)
Data and infrastructure: Airflow, Spark, Hive, Trino, feature stores, FastAPI, Docker, Kubernetes (in-house GPU platform for batch jobs and online serving), AWS
Evaluation: A/B testing, labeled-set benchmarks, Cohen's kappa, bootstrap confidence intervals
Web: React, Next.js, Vue.js, Node.js, Spring

Education

MS **KAIST AI** Mar. 2023 to Feb. 2025

- GPA: 3.94/4.3
- Advisor: James Thorne / Explainable Factual Reasoning Lab (xfact)
- Research: retrieval, retrieval-augmented generation (RAG), reward modeling for question answering

BS **Chung-Ang University**, Major: Computer Science and Engineering / Minor: Applied Statistics Mar. 2016 to Feb. 2023

- GPA: 3.98/4.5

Publications (First or Co-First Author on 4 of 5)

Learning to Insert [PAUSE] Tokens for Better Reasoning Jul. 2025
Findings of the Association for Computational Linguistics: ACL 2025
Eunki Kim*, Sangryul Kim*, James Thorne (*equal contribution)

Context Filtering with Reward Modeling in Question Answering Jan. 2025
In Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)
Sangryul Kim, James Thorne

ProbGate at EHRSQL 2024: Enhancing SQL Query Generation Accuracy through Probabilistic Threshold Filtering and Error Handling Jun. 2024
In Proceedings of the 6th Clinical Natural Language Processing Workshop at NAACL 2024
Sangryul Kim, Donghee Han, Sehyun Kim

Re3val: Reinforced and Reranked Generative Retrieval Mar. 2024
Findings of the Association for Computational Linguistics: EACL 2024
EuiYul Song, Sangryul Kim, Haeju Lee, Joonkee Kim, James Thorne

Implementing chatbot from K-drama: learning how to speak from the main characters Jun. 2022
Korea Computer Congress 2022 (KCC 2022, Domestic Conference)
Sangryul Kim, Seunguk Yu, Joonghoon Kim, Yoon-Sik Cho

Additional Experience

NAVER Boostcamp AI Tech, 3rd cohort, NAVER Connect Foundation (Jan. 2022 to Jun. 2022): Completed the NLP track, worked on text retrieval and named entity recognition, and took 1st place in the first team image classification competition on stages.ai.
Coding Academy Instructor, CIT Gangnam Apgujeong (Sep. 2021 to Mar. 2022): Taught Python, data science and machine learning classes and projects for adults, including students preparing for universities abroad.
Google Machine Learning Bootcamp, Google Developers (Aug. 2021 to Nov. 2021): Completed the Deep Learning Specialization by Andrew Ng on Coursera and took part in a Kaggle tabular competition.
SW Maestro, 8th cohort, Ministry of Science and ICT (Aug. 2017 to Dec. 2017): Developed Mononity, a tool that extracts and analyzes Android performance data through the ADB shell.

Awards

2nd Place: 2023 AI Graduate School Challenge, hosted by the Korean Ministry of Science and ICT / github

- Presented "Explainable and Customizable Multihop Question Answering", a RAG system with custom features built on Korean language models.

Excellent Paper Award: 2017 Fall KSII Conference

- "Analysis of Emotion Prediction Model by Genre Using Musical Features"
  Sangryul Kim, Hyunki Lim, Dae-Won Kim